#C Compare the two tags with a constant-time equality check.
#D If the tag is valid then call the real token store with the original token ID.

GENERATING THE KEY

The key used for HMAC-SHA256 is just a 32-byte random value, so you could just generate one using a `SecureRandom` just like you currently do for database token IDs. But many APIs will be implemented using more than one server to handle load from large numbers of clients, and requests from the same client may be routed to any server, so they all need to be using the same key. Otherwise, a token generated on one server will be rejected as invalid by a different server with a different key. Even if you have only a single server, if you ever restart it, then it will reject tokens issued before it restarted unless the key is the same. To get around these problems, you can store the key in an external *keystore* that can be loaded by each server.

> **DEFINITION** A ***keystore*** is an encrypted file that contains cryptographic keys and TLS certificates used by your API. A keystore is usually protected by a password.

Java supports loading keys from keystores using the `java.security.KeyStore` class, and you can create a keystore using the `keytool` command shipped with the JDK. Java provides several keystore formats, but you should use the PKCS #12 format (https://tools.ietf.org/html/rfc7292) because that is the most secure option supported by keytool.